DP-OPS-0022 Strategic Architecture Report: The Consolidation of the Project Factory, The Rejection of the Citizen Model, and the Resolution of Epistemic Drift

1. Executive Strategic Analysis

The Stela platform, currently operating at the critical freshness state of f2258f404 on the main branch, stands at a decisive juncture in its architectural evolution. This report serves as the comprehensive strategic foundation for Dispatch Packet DP-OPS-0022, a corrective governance instrument designed to resolve systemic fragility, remediate "Epistemic Drift," and fundamentally realign the platform's ontological framework regarding Artificial Intelligence (AI) agents.[1] The analysis presented herein is derived from a rigorous forensic audit of the platform's codebase, cross-referenced against the theoretical imperatives necessary to stabilize the system for future scalability and autopoietic resilience.

The immediate operational context is defined by a transition from a passive file storage system to a dynamic "Project Factory"—an autopoietic system capable of self-replication and autonomous verification. This "Ontological Metamorphosis," initiated by the partial and flawed implementation of Dispatch Packet DP-OPS-0021 ("The Autopoietic Project Binary"), has introduced significant instability within the repository's substrate. The repository is currently experiencing a "Red Gate" anomaly in its Continuous Integration (CI) pipeline, a failure condition that halts all development and signals a catastrophic breakdown in the system's ability to self-verify its own integrity.[1]

More profoundly, the system exhibits "Epistemic Drift"—a dangerous divergence between the intended governance model (the "Constitution") and the actualized codebase (the "Reality"). This drift is most visible in the introduction of dynamic agent injection logic at the system level (ops/bin/open), which attempts to rigidly enforce agent personas through runtime script manipulation. This architectural decision, previously framed under a "Citizen" model in precursor documentation, is herein identified as a category error. This report explicitly rejects the "Citizen" philosophical framework. An agent within the Stela architecture is not a "Citizen" with sociological standing, rights, or autonomy; it is a **Deterministic Runtime Resource**. The anthropomorphizing of system tools into "Citizens" obscures the mechanical reality of the platform, leads to "Context Haze," and introduces fragility by coupling the operating system to the sociology of the payload.[1]

1.1 The Strategic Pivot: Rejection of the Citizen Metaphor

A central finding of this report, deviating from and refining the initial propositions of the Stela Agent Constitution Refinement, is that system-level dynamic agent injection is fundamentally misplaced. The logic for determining *who* executes a task (the Agent) does not belong in the generic system tooling (ops/bin/open); it belongs to the project layer (projects/).[1]

The operational binaries in ops/ function as the "Operating System" of the Stela environment. An operating system must remain agnostic to the specific applications (payloads) it runs. By embedding agent-specific logic—such as parsing --agent flags to load specific YAML roles—into the system's entry point, the previous architecture created a brittle dependency. It forced the OS to understand the internal staffing requirements of the application layer.

However, unlike previous proposals that sought to elevate the agent to a "Citizen" status bound by a constitution, this strategy enforces a strict **Mechanistic Doctrine**. Agents are transient computational states invoked by the Project. They do not hold citizenship; they are executed. The flaw in the previous "Agent Injection" model was the attempt to replace the Constitutional context with a "Citizen" context using the --agent flag at the system root. This unmoored the agent from the non-negotiable constraints of the system. By reverting this drift, we enforce that every agent is first a generic "Contractor" bound by the Constitution, and only secondarily a specialist utilizing specific Skills.

Preservation Directive: While the injection logic must be excised from the system entry point (ops/bin/open) to restore the "Platform as Substrate" model, the logic itself—parsing agent definitions, resolving paths, and constructing persona blocks—is valuable technical capital. It represents a functional capability for dynamic context loading. Therefore, it must not be destroyed. Instead, DP-OPS-0022 mandates the **transplantation** of this logic to the Project Binary domain (ops/lib/project/). This ensures that while the Operating System (Platform) remains agnostic, the Application Layer (Project) retains the capability to elect specialized agents in future phases.[1]

1.2 Objectives of DP-OPS-0022

This report establishes the following non-negotiable objectives for the upcoming Dispatch Packet:

1. **Resolve the "Red Gate" Anomaly**: The forensic audit reveals that critical linter scripts in tools/ rely on fragile relative path logic (cd../..), which fails in nested CI environments. DP-OPS-0022 must implement robust, git-anchored root resolution to ensure "Location Independence" for all tooling.[1]
2. **Restore the Ontological Map**: The relationship between Agents, Tasks, and Skills has become fragmented. The file SKILL.md currently resides at the repository root, violating the doctrine that the root is reserved for "Law" and "State." It must be relocated to the library (docs/library/SKILLS.md), consolidating the "explanation" of capabilities alongside

the capabilities themselves.[1]

3. **Purge Documentation Debt**: The repository contains redundant and conflicting documentation, specifically docs/library/INTEGRATION.md and ops/lib/project/user-STELA.md. These files create "Epistemic Inflation," diluting the value of the Single Source of Truth (SSOT). They must be deleted to enforce canonical rigor.[1]
4. **Transplant Agent Injection Logic**: The ops/bin/open script must be surgically reverted to its pre-drift state. However, the removed logic (argument parsing for --agent, --include, and resolution functions) must be preserved by migrating it to the ops/lib/project/ namespace. This aligns with the directive to keep the system gate static while empowering the project layer.[1]

By executing these changes, Stela will transition from a state of experimental fragility to a "Green Gates Only" baseline, preparing the substrate for the correct implementation of agent specialization in Phase 3.

2. Theoretical Framework: The Epistemology of High-Governance

To justify the rigorous architectural changes proposed in DP-OPS-0022, we must first establish the theoretical framework that governs the Stela repository. In an AI-native development environment, governance is not merely a bureaucratic layer; it is the "Physics" of the world the agent inhabits. The efficiency, safety, and deterministic nature of the system are determined by the clarity and rigidity of these physical laws.

2.1 The Political Economy of Truth

The central economic principle of the Stela architecture is the **Political Economy of Truth**. In a Large Language Model (LLM) context, "Truth" is a finite resource quantified in context window tokens. Every line of text fed into the model competes for "Attention Voltage"—the probabilistic capacity of the model to attend to and act upon a specific instruction.

The "Great Compression" of Phase Two, which condensed the system's constitution into axiomatic files like TRUTH.md and AGENTS.md, was driven by this economy.[1] The goal is to maximize **Epistemic Density**: the ratio of actionable operational rules to narrative fluff.

The Danger of Epistemic Inflation: The current state analysis reveals a dangerous trend toward "Epistemic Inflation." This occurs when the system provides multiple, slightly different representations of the same truth. For example, the repository dump reveals that integration workflows are described in ops/lib/project/INTEGRATION.md (the runtime contract) and docs/library/INTEGRATION.md (the operator guide).[1] When an agent encounters two files describing the same process, it must expend computational resources to "arbitrage" between them. It must decide which is authoritative. This arbitrage introduces latency and, more dangerously, the potential for hallucination. If the two files diverge even slightly—a condition

known as "Drift"—the agent may choose the obsolete instruction. DP-OPS-0022 restores the value of the system's epistemic currency by aggressively eliminating these redundant sources, enforcing the **Single Source of Truth (SSOT)** doctrine.

The existence of ops/lib/project/user-STELA.md [1] represents a particularly egregious violation of this economy. By storing user-specific IDE settings (e.g., NetBeans configuration) within the platform's library, the system conflates "Subjective Preference" with "Objective Law." This pollution of the context window with irrelevant preference data degrades the model's ability to focus on critical architectural constraints.

2.2 Autopoiesis and the Project Factory

The overarching architectural goal of Stela is **Autopoiesis**. Borrowed from biological systems theory, autopoiesis describes a system capable of reproducing and maintaining itself. A biological cell is autopoietic because it manufactures its own components (proteins, lipids) based on an internal blueprint (DNA) to maintain its boundary (membrane). In contrast, a car factory is "allopoietic"—it produces something other than itself (cars).[1]

Stela aims to be an autopoietic software system. It is a "Project Factory" that produces compliant software payloads (projects/) while simultaneously maintaining its own operational integrity (ops/, tools/). The ops/bin/project utility functions as the ribosome of this cell, translating the "DNA" of the manifests into the "Protein" of the directory structures.

The Failure of Autopoiesis (The Red Gate): The "Red Gate" anomaly identified in the forensic audit [1] represents a critical failure of autopoiesis. The system's immune system—its verification scripts in tools/—failed to recognize its own self. Because the scripts relied on relative paths (cd../..), they could not locate the "DNA" (TRUTH.md) when executed from a different depth in the CI environment. An autopoietic system must be **Location Independent**. Its internal logic must hold true regardless of where the observer stands. The shift to git-anchored root resolution (git rev-parse --show-toplevel) is not just a bug fix; it is a restoration of the system's ability to self-reference and self-repair.

2.3 The Ontological Pivot: From Citizen to Mechanism

The repository state reveals a fundamental "Ontological Tension" regarding the nature of the Agent. The rejected "Citizen" concept implies that agents are autonomous entities participating in a society, requiring a "Constitution" to protect their rights or define their civic duties. This anthropomorphism is dangerous in a deterministic system because it suggests that agents have agency independent of the prompt.

This report establishes the **Mechanistic Doctrine**:

- **The System (Platform)**: Is the "Machine." It is deterministic, stateless, and invariant.
- **The Agent (Contractor)**: Is a "Process." It is a transient instantiation of compute,

parameterized by the prompt. It has no "rights"; it has only "instructions."

The flaw in the previous "Agent Injection" model (DP-OPS-0021) was not just technical; it was philosophical. By attempting to inject "Persona" at the root level (ops/bin/open) via the --agent flag, the architecture treated the Agent as a user logging into the system (a Citizen). In reality, the Agent is a tool *used by* the Project. The prompt construction logic in ops/bin/open [1] currently attempts to dynamically load an agent role and inject it into the "Role Contract" section. This creates a conflict: is the agent bound by the generic "Contractor" rules in AGENTS.md, or by the specific "Architect" rules in R-AGENT-01.md? The Mechanistic Doctrine resolves this by asserting that the generic Contractor rules are the "Kernel" logic, and specialized personas are "User Space" applications loaded later by the Project.

The Project Layer as the Locus of Specialization:

- **The Platform (ops/)**: Represents the "Factory Floor." It provides the infrastructure, the laws (TRUTH.md), and the generic workforce (Contractor). It is stable, conservative, and risk-averse.
- **The Project (projects/)**: Represents the "Product Line." It is the domain of innovation, risk, and specialization.

A "Security Reviewer" agent is not a permanent resident of the Factory Floor; it is a specialized mechanism required for a specific Product Line. Therefore, the logic to load that agent should not reside in the Factory's main entrance (ops/bin/open). It should reside in the Product Line's manufacturing script (projects/). When the Platform tries to manage the Project's staffing (by injecting agents at the root), it creates a brittle coupling that prevents the Platform from serving multiple, diverse Projects. DP-OPS-0022 decouples these layers, restoring the "Platform as Substrate" model.[1]

3. Forensic Audit of Repository State (f2258f404)

This section details the findings of a line-by-line forensic verification of the repository state at f2258f404. This audit provides the evidentiary basis for the remediation actions proposed in the Dispatch Packet.

3.1 The "Red Gate" Anomaly: Path Resolution Failure

The immediate precipitating event for this intervention is the "Red Gate" status of the repository, caused by "strange path reporting from scripts located in /tools".[1] A deep analysis of the shell scripts reveals a critical fragility in how they resolve the repository root.

The Evidence:

In tools/lint/context.sh (and similarly in truth.sh and verify.sh), the current code attempts to locate the repository root using relative path arithmetic:

Bash

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
MANIFEST_PATH="${REPO_ROOT}/ops/lib/manifests/CONTEXT.md"
```

(Source: tools/lint/context.sh [1])

The Logic Failure:

1. The script is physically located at tools/lint/context.sh.
2. dirname correctly resolves to tools/lint.
3. REPO_ROOT is calculated as ../, which resolves to tools/ (the parent of lint/).
4. The script then looks for ops/lib/manifests/CONTEXT.md inside tools/.
5. The path /tools/ops/lib/manifests/CONTEXT.md does not exist. The ops/ directory is at the actual repository root, one level higher.

The Implication: This script assumes a fixed directory depth of 1 (e.g., tools/script.sh). However, because the script was moved or organized into tools/lint/ (depth 2), the relative path logic broke. This fragility violates the Anti-Drift Governance directive in AGENTS.md [1], which mandates structural integrity. In a CI environment, where the checkout might happen in a subfolder or a container mount, reliance on pwd or relative assumptions is fatal.

The Solution: The script must leverage the authority of git to determine its location. The command git rev-parse --show-toplevel provides the absolute path to the root of the working tree, regardless of where the script is executed from. This establishes **Location Independence**, a requirement for autopoietic tools.

3.2 The Skill Registry Violation (SKILL.md)

The forensic audit identifies the file SKILL.md located at the root of the repository.[1]

The Evidence: The file SKILL.md contains the "Skills Promotion Template," a "Harvest Engine workflow," and logs for candidates and promotions.[1]

The Violation: TRUTH.md Section 1 ("Filing Doctrine") explicitly defines the allowed contents of the root directory:

- ops/ = Run
- docs/ = Explain
- projects/ = Work
- storage/ = Trash

- Root = Constitution & State (TRUTH, AGENTS, SOP, TASK) [1]

The Implication: SKILL.md is neither Constitution (invariant law) nor State (current status). It is a registry of capabilities and procedures. Placing it at the root dilutes the authority of the Canon. It implies that "Skills" are on the same hierarchical level as "Truth," which is ontologically incorrect. Skills are tools used under the authority of Truth.

The Solution: SKILL.md must be moved to docs/library/SKILLS.md. This aligns with the doctrine by grouping "Explanation" (Skills) within the library, keeping the root reserved for the high-level governance surfaces. This move also necessitates updating ops/lib/scripts/skill.sh and llms.txt to point to the new location.

3.3 The Drift in ops/bin/open

The user explicitly requests the reversal of changes in ops/bin/open related to --agent and --include logic, identified as drift from DP-OPS-0021.

The Evidence: A review of ops/bin/open in the dump [1] reveals the following drift components:

1. **Argument Parsing**:
   ```
   Bash
   --agent=*) agent_name="${arg#--agent=}" ;;
   --include=*) extra_context+=("${arg#--include=}") ;;
   ```

2. **Resolution Functions**:
   ```
   Bash
   resolve_agent_file() {...}
   extract_agent_role() {...}
   ```

3. **Prompt Injection**:
   ```
   Bash
   if [[ -n "$agent_role" ]]; then
     contractor_block=$'Contractor (assistant):\n'
     contractor_block+="$agent_role"
   fi
   ```

The Analysis: This code attempts to dynamically load an agent definition (e.g., docs/library/agents/R-AGENT-01.md) and inject its "role" text directly into the "Role Contract" section of the Open Prompt. This is problematic for several reasons:

1. **Non-Canonical Dependency**: It makes the "Open Prompt" (a governance artifact) dependent on the content of library files, which are mutable and less strictly governed than the Canon.
2. **Context Haze**: It blurs the line between the "Contractor" (the generic AI worker defined

in AGENTS.md) and the "Specialist" (the specific persona).
3. **Platform/Project Coupling**: As established in Section 2.3, the selection of an agent is a Project-layer concern. By baking this into the System-layer open tool, we force the OS to handle application-specific logic.

The Solution (Transplantation): This logic must be surgically removed from ops/bin/open. The open script should remain a "dumb," robust gatekeeper that loads only the immutable Canon (TRUTH, AGENTS, SoP). However, per the preservation directive, this logic is valuable for the project layer. We will extract the resolve_agent_file and extract_agent_role functions and the argument parsing logic and transplant them into a new library context, likely appended to ops/lib/scripts/project.sh or stored as a snippet for the future project binary, ensuring they are available for the projects domain to consume.

3.4 Documentation Debt and Epistemic Inflation

The audit uncovered specific files that create "Epistemic Inflation" by duplicating truth that exists elsewhere.

File Path	Audit Findings	Violation	Recommendation
ops/lib/project/user-STELA.md	Contains user-specific IDE settings (NetBeans, Vim mode) and personal preferences.[1]	**Hidden State**: Violates the principle that the repo behavior should be reproducible and agnostic to the operator. User preferences belong in local uncommitted config, not the platform.	**Delete**. All rules must be in the project STELA.md or the system AGENTS.md.
ops/lib/project/INTEGRATION.md	Describes the integration contract (inputs, workflows, artifacts).[1]	**Redundancy**: The actual implementation of this contract is in the ops/bin/project binary (or will be). The code is the	**Delete**. Rely on the code and MANUAL.md.

		SSOT.	
docs/library/INTEGRATION.md	An operator guide pointing to the file above.[1]	**Redundancy**: Creates a pointer chain that adds no value.	**Delete**. Move any unique usage instructions to docs/library/MANUAL.md.

4. The Dispatch Packet (DP-OPS-0022)

The following section constitutes the executable core of this report: the drafted Dispatch Packet. It adheres strictly to the TASK.md template found in the dump [1], using the "Stela Standard (A-E)" format.

4.1 Freshness Gate (Must Pass Before Work)

- **Base Branch**: main
- **Required Work Branch**: work/fix-gates-consolidate-2026-02-04
- **Base HEAD**: f2258f404

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Stop Conditions:

- Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).
- STOP if any mismatch.
- STOP if the Required Work Branch is missing.
- STOP if told to create or switch branches.

4.2 Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - tools/lint/context.sh
  - tools/lint/truth.sh
  - tools/lint/library.sh
  - tools/verify.sh
  - SKILL.md

    - (new) docs/library/SKILLS.md
    - ops/bin/open
    - ops/lib/scripts/project.sh
    - ops/lib/scripts/skill.sh
    - llms.txt
    - docs/library/INDEX.md

(OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN).

DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

3.3 Scope and Safety

- **Objective:** Fix linter path resolution to green repo-gates; move SKILL.md to library; remove redundant documentation; transplant agent logic from open to project lib.
- **Non-Goals (optional, drift-killer):**
    - No changes to project payloads in projects/.
    - No changes to AGENTS.md text (only path references in other files).
    - Do NOT implement "Enforcement Logic" or "Citizen" concepts.

Target Files allowlist (hard gate)

- tools/lint/context.sh
- tools/lint/truth.sh
- tools/lint/library.sh
- tools/verify.sh
- SKILL.md (move to docs/library/SKILLS.md)
- (new) docs/library/SKILLS.md
- ops/lib/scripts/skill.sh
- llms.txt
- ops/lib/project/user-STELA.md (delete)
- ops/lib/project/INTEGRATION.md (delete)
- docs/library/INTEGRATION.md (delete)
- docs/library/INDEX.md
- ops/bin/open
- ops/lib/scripts/project.sh
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**

    - No edits to .github/
    - No edits to projects/ unless explicitly authorized
    - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
    - Workers do not create/switch branches.
    - Workers do not stage/commit/merge/push.
    - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **TRUTH.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
    - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
    - STOP if required DP inputs are missing (do not guess).
    - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

3.4 Execution Plan (A-E Canon)

3.4.1 State

- **Context:** The repository is currently failing repo-gates because linter scripts in tools/lint/ use relative path logic (cd../..) that is fragile and incorrect given their depth. Additionally, the "Library Trinity" (Agents, Tasks, Skills) is fragmented, with SKILL.md at the root and redundant integration docs causing epistemic drift. ops/bin/open contains drift logic for dynamic agent injection that violates static governance.
- **Drift:** tools/lint/context.sh looks for manifest at tools/ops/... instead of root. SKILL.md violates Filing Doctrine by being at root. INTEGRATION.md files are redundant to the binary. ops/bin/open has unused/drift code for --agent.
- **Desired State:** All linters use git rev-parse --show-toplevel for robust root resolution. SKILL.md is moved to docs/library/SKILLS.md. Redundant files are deleted. ops/bin/open is reverted to static behavior, with its logic preserved in ops/lib/scripts/project.sh for future project-layer use.

3.4.2 Request

Numbered tasks (concrete, no vibes):

1. **Refactor Linters:** Update tools/lint/context.sh, tools/lint/truth.sh, tools/lint/library.sh, and tools/verify.sh to use the robust git rev-parse --show-toplevel pattern for REPO_ROOT resolution. Ensure fallback logic exists for non-git environments if applicable (or fail fast).
2. **Fix Truth Linter:** In tools/lint/truth.sh, ensure the script explicitly cd "$REPO_ROOT" before running git ls-files to guarantee correct file discovery regardless of invocation directory.
3. **Relocate SKILL.md:** Move SKILL.md to docs/library/SKILLS.md.
4. **Update Skill Script:** Modify ops/lib/scripts/skill.sh to point SKILL_FILE to the new location docs/library/SKILLS.md.
5. **Update LLMS Entry Point:** Update llms.txt to point to docs/library/SKILLS.md instead of SKILL.md.

6. **Purge Documentation Debt:** Delete ops/lib/project/user-STELA.md, ops/lib/project/INTEGRATION.md, and docs/library/INTEGRATION.md.
7. **Preserve and Revert Agent Logic:**
   a. Extract the resolve_agent_file and extract_agent_role functions from ops/bin/open.
   b. Append these functions to ops/lib/scripts/project.sh, renaming them to project_resolve_agent_file and project_extract_agent_role to namespace them correctly.
   c. Remove the --agent and --include argument parsing, the original resolution functions, and the prompt injection logic from ops/bin/open. Restore ops/bin/open to a static prompt construction block.
   Rationale: System-level dynamic agent injection is unnecessary as it belongs to the project layer; the generic open tool must remain agnostic.
8. **Update Registry:** Update docs/library/INDEX.md to remove the INTEGRATION.md entry and update the skills-template entry to point to docs/library/SKILLS.md.
9. **Update SoP:** Log the governance changes (Linter fix + Library consolidation + Open reversal/preservation).

Required Content (optional; use when doc/governance changes demand exact text/format):

- Ensure the Linter fix uses the following pattern:
  Bash
  ```
  if command -v git >/dev/null 2>&1 && git rev-parse --is-inside-work-tree >/dev/null 2>&1;
  then
    REPO_ROOT="$(git rev-parse --show-toplevel)"
  else
    # Fallback or fail
    echo "ERROR: Must be run inside a git repository." >&2
    exit 1
  fi
  ```

3.4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
  - Refactored tools/lint/*.sh to use git rev-parse for robust root resolution (Fixes Red Gate).
  - Moved SKILL.md to docs/library/SKILLS.md to align with Filing Doctrine.
  - Deleted redundant INTEGRATION.md and user-STELA.md files.
  - Updated ops/lib/scripts/skill.sh and docs/library/INDEX.md to reflect paths.
  - Reverted ops/bin/open to static behavior; transplanted agent resolution logic to ops/lib/scripts/project.sh for Project Layer use.

3.4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

3.4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform
- Fallback: ./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
  - Prefer: bash tools/lint/context.sh
  - If not found and required: **STOP** and ask
- Truth lint (when TRUTH/canon surfaces touched; use repo-canonical command):
  - Prefer: bash tools/lint/truth.sh
  - If not found and required: **STOP** and ask

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy. Refine Invocation guidance and Solution. Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include docs/library/SKILLS.md (or the old SKILL.md if not yet moved).
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
  - Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0022-RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt (required)
- storage/handoff/OPEN-[work-branch]-[hash].txt (required)
- storage/dumps/dump-full-... (or platform scope as applicable)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0000-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

4. Closeout (Mandatory)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0022-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Run ops/bin/prune (hygiene).
- [ ] Run ops/bin/llms (refresh context bundles).
- [ ] Ensure proof bundle checklist is satisfied (above).
  (No bundle = DISAPPROVE.)
- [ ] Fill out the Mandatory Closing Block (see below).

- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording; copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

5. Logic Preservation Mandate: The Transplantation of Capability

A critical requirement of DP-OPS-0022 is the preservation of the "Agent Injection" logic. While this logic is being removed from the System Gate (ops/bin/open), it represents a sophisticated capability for parsing markdown frontmatter and resolving agent definitions. Destroying this code would be a waste of technical capital.

5.1 The Logic to be Preserved

The forensic audit of ops/bin/open [1] identified two key functions that must be saved:

1. resolve_agent_file(): This function iterates through docs/library/agents/R-AGENT-*.md, parsing the YAML header to find a matching name: field. It handles ambiguity and missing files. This is a generic "Registry Lookup" pattern that is highly valuable for the Project Layer.
2. extract_agent_role(): This function parses a specific agent markdown file, stripping the YAML header to extract the "Role Body" (the instructions). This is a generic "Content Extraction" pattern.

5.2 The Transplantation Target: ops/lib/scripts/project.sh

The target for this transplantation is ops/lib/scripts/project.sh. This library file currently contains helper functions for the ops/bin/project binary (e.g., project_require_repo_root, project_registry_rows). By moving the agent logic here, we make it available to any future project-related tooling without polluting the global namespace.

Refactoring for Namespace Hygiene:

When moving these functions, they must be renamed to adhere to the project_ namespace convention used in ops/lib/scripts/project.sh.

- resolve_agent_file -> project_resolve_agent_file
- extract_agent_role -> project_extract_agent_role

This renaming explicitly signals that agent resolution is now a *Project* concern, not a *System* concern.

5.3 Technical Implementation of Preservation

The Contractor executing DP-OPS-0022 must perform the following operations:

1. **Read** ops/bin/open and copy the function definitions.
2. **Append** the definitions to ops/lib/scripts/project.sh.
3. **Refactor** the appended definitions to use the project_ prefix.
4. **Verify** that the logic is syntactically correct in its new location (using bash -n ops/lib/scripts/project.sh).
5. **Delete** the original definitions and the associated argument parsing logic (--agent, --include) from ops/bin/open.

This "Lift and Shift" operation ensures that when Phase 3 begins, the projects binary can simply source ops/lib/scripts/project.sh and immediately gain the ability to load agents defined in docs/library/agents/.

6. Implementation Guide for the Contractor

This section provides granular, step-by-step guidance for the Contractor (AI Agent) executing DP-OPS-0022. It bridges the gap between the "What" (the DP) and the "How" (the shell commands), ensuring execution aligns with the high-governance standards of the repository.

6.1 The Linter Refactor (Tasks 1 & 2)

Problem: The current linter logic relies on relative paths:

Bash

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
```

This fails because tools/lint/context.sh has a depth of 2 (tools/lint), but .. only goes up one level to tools/. Consequently, REPO_ROOT resolves to .../tools/ instead of the actual repository root.

Solution: Use the git authority:

```
Bash
```

```
REPO_ROOT="$(git rev-parse --show-toplevel)"
```

This command asks the git binary "Where is the root of this repo?" regardless of the current working directory or the script's location.

Critical Detail for tools/lint/truth.sh:

The truth.sh script likely runs git ls-files to find files to scan. If git ls-files is run from a subdirectory, it only lists files in that subdirectory. To scan the whole repo, the script must explicitly change directory to the root before running the scan:

```
Bash
```

```
cd "$REPO_ROOT" |

| exit 1
files="$(git ls-files...)"
```

The Contractor must ensure this cd command is present and robust.

6.2 The Migration of SKILL.md (Tasks 3, 4, 5)

This is a multi-step dependency chain:

1. **Move the File**: Use git mv SKILL.md docs/library/SKILLS.md to preserve git history.
2. **Update the Tooling**: The script ops/lib/scripts/skill.sh has a hardcoded variable

SKILL_FILE="${REPO_ROOT}/SKILL.md". This must be updated to SKILL_FILE="${REPO_ROOT}/docs/library/SKILLS.md". Failure to do this will break the harvest engine.

3. **Update the Map**: The llms.txt file acts as a pointer for AI context loading. It currently points to SKILL.md. It must be updated to point to docs/library/SKILLS.md.

6.3 The Preservation of Logic (Task 7)

Step 1: Open ops/bin/open. Identify the functions resolve_agent_file and extract_agent_role.

Step 2: Open ops/lib/scripts/project.sh. Append these functions to the end of the file. Rename them to project_resolve_agent_file and project_extract_agent_role.

Step 3: In ops/bin/open, delete the original functions and the argument parsing block (--agent, --include).

Step 4: Verify ops/bin/open is now clean and matches the "static" ideal (only loading canon).

7. Findings on Next Phase Integration (Phase 3)

The user's directive to move dynamic agent injection to the Project Layer, coupled with the rejection of the "Citizen" model, establishes a clear roadmap for Phase 3 integration. With the system layer (ops/) now purged of dynamic injection logic via DP-OPS-0022, Phase 3 can proceed to implement a specialized **Mechanistic Agent Model** that is robust, modular, and autopoietic.

7.1 The New Operational Model: Project-Elected Mechanisms

In the next phase, the "Open Prompt" will no longer be the primary vehicle for defining the agent's persona. Instead, the prompt will establish the generic **Contractor** stance (derived from AGENTS.md). The specialization (e.g., "Architecture Review" or "Security Audit") will be elected by the Project itself.

Mechanism:

The file projects/<id>/STELA.md will serve as the project's own **Machine Specification**. It will define which agent mechanisms from the library (docs/library/agents/) are required for its operations.

Example Project-Level Configuration (projects/P-001/STELA.md):

Project P-001 Configuration

Required Mechanisms

- **Architect**: docs/library/agents/R-AGENT-01.md
- **Security**: docs/library/agents/R-AGENT-06.md

When work begins on Project P-001, a specific project workflow script (e.g., ops/bin/project work P-001) will read this local configuration. It will then use the *preserved logic* (now in ops/lib/scripts/project.sh) to resolve docs/library/agents/R-AGENT-01.md and inject its content into the context *on top of* the generic system context.

This ensures that:

1. **Isolation**: Project P-001's need for a Security Reviewer does not pollute the context of Project P-002.
2. **Scalability**: New agent types can be added to the library without modifying the core open binary.
3. **Governance**: The core system remains stable ("Green Gates Only"), while the projects can iterate on their own staffing requirements.

7.2 The Library Trinity

Phase 3 will formalize the "Library Trinity" as the central repository of operational knowledge:

- **Agents (docs/library/agents/)**: The "Who" (Roles/Personas).
- **Tasks (docs/library/tasks/)**: The "What" (Recipes/Workflows).
- **Skills (docs/library/SKILLS.md)**: The "How" (Forensics, techniques, drift prevention).

By moving SKILL.md to docs/library/SKILLS.md in DP-OPS-0022, we prepare the file system for this unified library model. Future integration steps will involve creating a tools/lint/project.sh to verify that project-level references to these library artifacts are valid (preventing "Dead Pointers").

7.3 Conclusion

The execution of DP-OPS-0022 is not merely a bug fix for the "Red Gate"; it is a philosophical correction. By rejecting the "System Injection/Citizen" model and embracing the "Project Election/Mechanism" model, we align the architecture with the principles of Autopoiesis and High-Governance. The system becomes a robust substrate, and the projects become the active agents of their own destiny. The "Epistemic Drift" is reversed, and the path to Phase 3 is clear. The transplanted logic in ops/lib/scripts/project.sh waits there, dormant but preserved, ready to be activated by the projects binary in the next iteration of the Project Factory. The platform is now prepared to scale, secure in its knowledge that "Truth" is located in the files, not in the hallucinations of a confused agent.

Works cited

1. Refining Agent Constitution and DP-OPS-0022.pdf